



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

There are combinations of the chess that is unimaginably large and also, the agent should store every percept. as the lifetime increases the worst case will be very massive table to store the percept sequence that the physical hardware can not store. So there is no way to create a perfect mathematically table driven agent for chess.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

1. Each if is a condition and each return is the action for that condition.
2. IF food_here THEN Stay = condition-action rule (see food, eat it, no other thinking).
3. IF wall_ahead THEN turn_left = another rule (wall in front, turn).
4. ELSE move_forward = default rule when no condition matches.

so the agent looks at current percept only, matches it to a rule and do the action. No memory used, no planning.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

How the partial observability works here is, the percept only tells about the current cell and the one cell ahead (wall_ahead, food_here). agent cannot see the whole map, cannot see cells behind it. and also agent has no `__init__`, so it cannot remember past positions or past actions. every decision is based only on this one moment's percept. so there is no percept history also.

so agent hits wall, turns left. next percept again shows wall, turns left again. since it does not remember it already turned left here before, it just keeps repeating the same turn pattern.

as the result, same sequence of percepts repeats, same actions repeat.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

```
def _peek(self, direction):  
    x, y = self.rel_pos  
    if direction == 'Up':  
        y += 1
```

The above code predicts next position if agent moves in given direction.

```
self.rel_pos = self._peek(action)  
self.visited_cells.add(self.rel_pos)  
self.last_action = action
```

after deciding action, agent updates its internal belief of where it is now (rel_pos) and adds that cell to memory (visited_cells), and also updated for edge blocks (blocked_ahead = percept['wall_ahead'] or self._at_edge(facing)).

Here _at_edge also part of world model, since the percept alone doesn't tell about map boundary, so agent's own model fills that gap.

so the order is first sense current percept. next check condition against own memory (left_is_visited), next pick rule/action, then update internal model. and the next loop starts with new updated state.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING